Migrating to DynamoDB from a relational database

Migrating a relational database into DynamoDB requires careful planning to ensure a successful outcome. This guide will help you understand how this process works, what tools you have available, and then how to evaluate potential migration strategies and select one that'll fit your requirements.

Topics

- Reasons to migrate to DynamoDB
- Considerations when migrating a relational database to DynamoDB
- Understanding how a migration to DynamoDB works
- Tools to help migrate to DynamoDB
- Choosing the appropriate strategy to migrate to DynamoDB
- Performing an offline migration to DynamoDB
- Performing a hybrid migration to DynamoDB
- Performing an online migration to DynamoDB by migrating each table 1:1
- Perform an online migration to DynamoDB using a custom staging table

Reasons to migrate to DynamoDB

Migrating to Amazon DynamoDB presents a range of compelling benefits for businesses and organizations. Here are some key advantages that make DynamoDB an attractive choice for database migration:

- **Scalability**: DynamoDB is designed to handle massive workloads and scale seamlessly to accommodate growing data volumes and traffic. With DynamoDB, you can easily scale your database up or down based on demand, ensuring that your applications can handle sudden spikes in traffic without compromising performance.
- **Performance**: DynamoDB offers low-latency data access, enabling applications to retrieve and process data with exceptional speed. Its distributed architecture ensures that read and write operations are distributed across multiple nodes, delivering consistent, single-digit millisecond response times even at high request rates.
- **Fully managed**: DynamoDB is a fully managed service provided by AWS. This means that AWS handles the operational aspects of database management, including provisioning, configuration,

patching, backups, and scaling. This allows you to focus more on developing your applications and less on database administration tasks.

- **Serverless architecture**: DynamoDB supports a serverless model, known as DynamoDB on-demand, where you pay only for the actual read and write requests your application makes with no upfront capacity provisioning required. This pay-per-request model offers cost efficiency and minimal operational overhead, as you only pay for the resources you consume without the need to provision and monitor capacity.
- **NoSQL flexibility**: Unlike traditional relational databases, DynamoDB follows a NoSQL data model, providing flexibility in schema design. With DynamoDB, you can store structured, semi-structured, and unstructured data, making it well-suited for handling diverse and evolving data types. This flexibility enables faster development cycles and easier adaptation to changing business requirements.
- **High availability and durability**: DynamoDB replicates data across multiple availability zones within a Region, ensuring high availability and data durability. It automatically handles replication, failover, and recovery, minimizing the risk of data loss or service disruptions. DynamoDB provides an availability SLA of up to 99.999%.
- **Security and compliance**: DynamoDB integrates with AWS Identity and Access Management for fine-grained access control. It provides encryption at rest and in-transit, ensuring the security of your data. DynamoDB also adheres to various compliance standards, including HIPAA, PCI DSS, and GDPR, enabling you to meet regulatory requirements.
- **Integration with AWS Ecosystem**: As part of the AWS ecosystem, DynamoDB seamlessly integrates with other AWS services, such as AWS Lambda, CloudFormation, and AWS AppSync. This integration enables you to build serverless architectures, leverage infrastructure as code, and create real-time data-driven applications.

Considerations when migrating a relational database to DynamoDB

Relational database systems and NoSQL databases have different strengths and weaknesses. These differences make database design different between the two systems.

Type of task	Relational database	NoSQL database
Querying the database	In relational databases, data can be queried flexibly,	In a NoSQL database such as DynamoDB, data can be

Type of task	Relational database	NoSQL database
	but queries are relatively expensive and don't scale well in high-traffic situations (see First steps for modeling relational data in DynamoDB). A relational database application may implement business logic in stored procedures, SQL subqueries, bulk update queries, and aggregation queries.	queried efficiently in a limited number of ways, outside of which queries can be expensive and slow. Writes to DynamoDB are singletons. Application business logic that formerly ran in stored procedures must be refactored to run outside of DynamoDB in custom code running on a host such as Amazon Amazon EC2 or AWS Lambda.
Designing the database	You design for flexibility without worrying about implementation details or performance. Query optimization generally doesn't affect schema design, but normalization is important.	You design your schema specifically to make the most common and important queries as fast and as inexpensive as possible. Your data structures are tailored to the specific requirements of your business use cases.

Designing for NoSQL database requires a different mindset than designing for a relational database management system (RDBMS). For an RDBMS, you can create a normalized data model without thinking about access patterns. You can then extend it later when new questions and query requirements arise. You can organize each type of data into its own table.

With NoSQL design, you can design your schema for DynamoDB when you know the questions it'll need to answer. Understanding the business problems and the application read and write patterns is essential. You should also aim to maintain as few tables as possible in a DynamoDB application. Having fewer tables keeps things more scalable, requires less permissions management, and reduces overhead for your DynamoDB application. It can also help keep backup costs lower overall.

The task of modeling relational data for DynamoDB and building a new version of the front-end application is a separate topic. This guide assumes you have a new version of your application built to use DynamoDB, but you still need to determine how best to migrate and synchronize historical data during the cutover.

Sizing Considerations

The maximum size of each item (row) that you store in a DynamoDB table is 400KB. For more information, see the section called "Quotas". The item size is determined by the total size of all attribute names and attribute values in an item. For more information, see the section called "Item sizes and formats".

If your application needs to store more data in an item than the DynamoDB size limit permits, break the item into an item collection, compress the item data, or store the item as an object in Amazon Simple Storage Service (Amazon S3) while storing the Amazon S3 object identifier in your DynamoDB item. See the section called "Large items". The cost to update an item is based on the full size of the item. For workloads that require frequent updates to existing items, having small items of one or two KB will cost less to update than larger items. See the section called "Working with Item Collections" for more information on item collections.

When choosing the partition and sort key attributes, other table settings, item size and structure, and whether to create secondary indexes, be sure to review the DynamoDB Modeling documentation as well as the guide for the section called "Cost optimization". Be sure to test your migration plan so your DynamoDB solution is cost efficient and fits within DynamoDB's features and limitations.

Understanding how a migration to DynamoDB works

Before reviewing the migration tools available to us, consider how writes are processed by DynamoDB.

The default and most common write operation is a single `PutItem` API operation. You can perform a `PutItem` operation in a loop to process data sets. DynamoDB supports virtually unlimited concurrent connections, so assuming you can configure and run a massively multi-threaded loading routine such as MapReduce or Spark, the velocity of writes is only limited by the capacity of the target table (which is also generally unlimited).

When loading data into DynamoDB, it's important to understand your loader's write velocity. If the items (rows) you are loading are 1KB in size or less, this velocity is simply the number of items per second. The target table can then be provisioned with sufficient WCU (write capacity units) to handle this rate. If your loader exceeds the provisioned capacity in any given second, the extra requests may be throttled or rejected altogether. You can check for throttles in the CloudWatch charts found in the DynamoDB console monitoring tab.

The second operation that can be performed is with a related API called `BatchWriteItem`. `BatchWriteItem` allows you to combine up to 25 write requests into one API call. These are received by the service and processed as separate `PutItem` requests to the table. Currently, when choosing `BatchWriteItem`,you will not get the advantage of automatic retries that is included with the AWS SDK when making singleton calls with `PutItem`. So if there are any errors (such as throttling exceptions), you'll have to look for the list of any failed writes on the response call to `BatchWriteItem`. For more information on handling throttling warnings in case these are detected in the CloudWatch throttling charts, see the section called “Throttling”.

The third type of data import is possible with the DynamoDB Import from S3 feature. This feature allows you to stage a large dataset in Amazon S3 and ask DynamoDB to automatically import the data into a new table. The import is not instant and will take time proportional to the size of the dataset. However, it's convenient since it requires no ETL platform or custom DynamoDB code. DynamoDB loads the data into a new table that's created by the import. Currently, it doesn't allow you to load data into an existing table. DynamoDB imports the data as-is, with no transformations. Similar to `PutItem`, it requires an upstream process and writes the data in your chosen format to an Amazon S3 bucket.

Tools to help migrate to DynamoDB

There are several common migration and ETL tools you can use to migrate data into DynamoDB.

Amazon provides a host of data tools that can be used in migration, including AWS Database Migration Service (DMS), AWS Glue, Amazon EMR, and Amazon Managed Streaming for Apache Kafka. All of these tools can be used to perform a downtime migration, and they can leverage relational database Change Data Capture (CDC) features to support online migrations. When choosing a tool, it'll help to consider the skill set and experience your organization has with each tool along with the features, performance, and cost of each one.

Many customers choose to write their own migration scripts and jobs in order to build custom data transformations for the migration process. If you plan to operate a high volume DynamoDB

table with heavy write traffic or regular large bulk load jobs, you may wish to write migration code yourself to get more familiar with the behavior of DynamoDB under heavy write traffic. Scenarios such as throttle handling and efficient table provisioning can be experienced early in the project when performing a practice migration.

Choosing the appropriate strategy to migrate to DynamoDB

A large relational database application may span a hundred or more tables and support several different application functions. When approaching a large migration, consider breaking your application into smaller components or micro-services, and migrating a small set of tables at a time. You can then migrate additional components to DynamoDB in waves.

When selecting a migration strategy, various factors may steer you towards one solution or another. We can present these options in a decision tree to simplify the options available to us given our requirements and resources available. The concepts are briefly mentioned here (but will be covered in more depth later in the guide):

- **Offline migration**: if your application can tolerate some downtime during the migration, it'll simplify the migration process.
- **Hybrid migration**: this approach allows for partial uptime during a migration, such as allowing reads but not writes, or allowing reads and inserts but not updates and deletes.
- **Online migration**: applications that require zero downtime during migration are less easy to migrate, and can require significant planning and custom development. One key decision is to estimate and weigh the costs of building a custom migration process versus the cost to the business of having a downtime window during cutover.

If	And	Then
You're okay to take the application down for some time during a maintenance window to perform the data migration. This is an **offline migration**.		Use AWS DMS and perform an offline migration using a full load task. Pre-shape the source data with a SQL `VIEW` if desired.
You're okay to run the application in read-only mode		Disable writes within the application or source

If	And	Then
during migration. This is a **hybrid migration**.		database. Use AWS DMS and perform an offline migration using a full load task.
You're okay to run the application with reads and new record inserts, but no updates or deletes, during the migration. This is a **hybrid migration**.	You have application development skills and can update the existing relationa l app to perform dual writes including to DynamoDB, for all new records	Use AWS DMS and perform an offline migration using a full load task. Concurren tly, deploy a version of the existing app that allows reads and performs dual writes.
You require a migration with minimal downtime. This is an **online** migration.	• You're migrating source tables 1-for-1 into DynamoDB without major schema changes.	Use AWS DMS to perform an online data migration. Run a bulk load task followed by CDC sync task.
You require a migration with minimal downtime. This is an **online** migration.	• You're combining source tables into fewer DynamoDB tables following the stacked schema or single table philosophy. • You have backend database development skills and spare capacity on the SQL host.	Create the NoSQL-ready table within the SQL database. Populate and synchronize it using JOINs, UNIONs, VIEWs, triggers, stored procedures.
You require a migration with minimal downtime. This is an **online** migration.	• You're combining source tables into fewer DynamoDB tables following single table philosophy. For example: • You don't have backend database development skills and spare capacity on the SQL host.	Consider the hybrid or offline migration approaches.

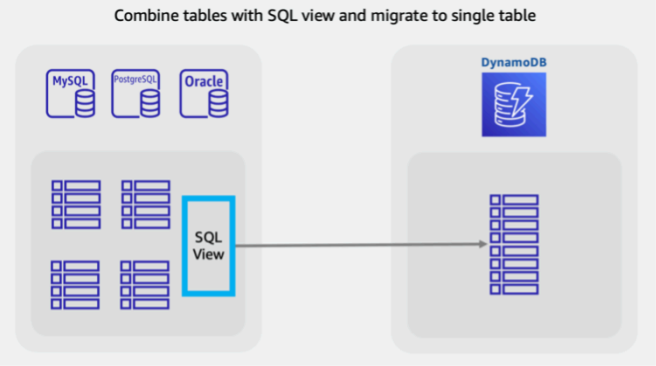
If	And	Then
You require a migration with minimal downtime. This is an **online** migration.	You're okay to skip migrating historical transaction data, or can archive it in Amazon S3 in lieu of migrating it. You just need to migrate a few small static tables.	Write a script or use any ETL tool to migrate the tables. Pre-shape the source data with a SQL `VIEW` if desired.

Performing an offline migration to DynamoDB

Offline migrations are suitable for when you can allow a downtime window to perform the migration. Relational databases commonly take at least some downtime each month for maintenance and patching, or may take longer downtimes for hardware upgrades or major release upgrades.

Amazon S3 can be used as a staging area during a migration. Data stored in CSV (comma separated values) or DynamoDB JSON format can be automatically imported into a new DynamoDB table using the DynamoDB import from S3 feature.

You may want to combine tables to leverage unique NoSQL access patterns (for example, transforming four legacy tables into a single DynamoDB table). A single key-value document request or a query for a pre-grouped item collection usually returns with better latency than a SQL database that performs a multi-table join. However, this makes the migration task more difficult. A SQL view could do the work within the source database to prepare a single dataset representing all four tables in one set.



This view can `JOIN` tables into a denormalized form, or could keep the entities normalized and stack tables using a SQL `UNION`. Key decisions around re-shaping relational data are covered in

this video. For offline migrations, using a view to combine tables is a great way to shape data for a DynamoDB single table schema.

Plan

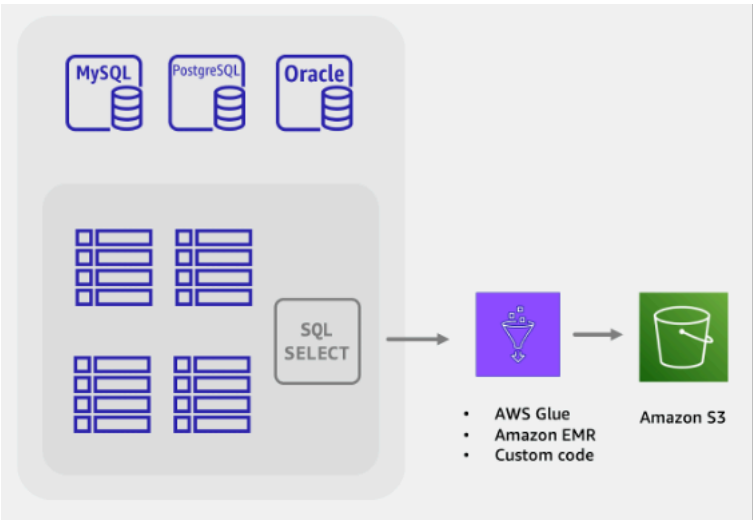
Perform an offline migration using Amazon S3

Tools

- An ETL job to extract and transform SQL data and store it in an S3 bucket such as:
  - AWS Database Migration Service, a service that can bulk load historical data and can also process Change Data Capture (CDC) records to synchronize source and target tables.
  - AWS Glue
  - Amazon EMR
  - Your own custom code
- The DynamoDB import from S3 feature

Offline migration steps:

1. Build an ETL job that can query the SQL database, transform table data into DynamoDB JSON or CSV format, and save it to an S3 bucket.



2. The DynamoDB Import from S3 feature is invoked to create a new table and automatically load data from your S3 bucket.

The fully offline migration is simple and straightforward, but it may not be popular with application owners and users. Users would benefit if the application could provide reduced levels of service during the migration, instead of no service at all.

You could add functionality to disable writes during the offline migration, while allowing reads to continue as normal. Application users could still safely browse and query for existing data while the relational data is being migrated. If this is what you're looking for, continue reading to learn about hybrid migrations.

Performing a hybrid migration to DynamoDB

While all database applications perform read and write operations, the types of write operations being performed should be considered when planning a hybrid or online migration. Database writes can be classified into three buckets: inserts, updates, and deletes. Some applications may not require immediate processing of deletions. These applications can defer deletions to a bulk cleanup process at the end of the month, for example. These types of applications can be simpler to migrate while allowing partial uptime.

Plan

Perform a hybrid online/offline migration with application dual writes

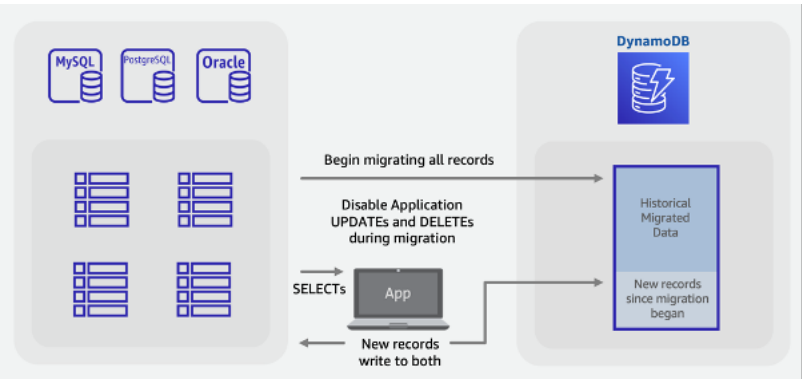
Tools

- An ETL job to extract and transform SQL data and store it in an S3 bucket such as:
  - AWS DMS
  - AWS Glue
  - Amazon EMR
  - Your own custom code

Hybrid migration steps:

1. Create the target DynamoDB table. This table will receive both historical bulk data and new, live data
2. Create a version of the legacy application that has deletes and updates disabled while performing all inserts as dual writes to both the SQL database and DynamoDB
3. Begin the ETL job or AWS DMS task to backfill existing data and deploy the new application version at the same time

4. When the backfill job completes, DynamoDB will have all existing and new records and be ready for application cutover



> **Note**
>
> The backfill job writes directly from SQL to DynamoDB. We are unable to use the S3 import feature as in the offline migration example, since that feature creates a new table that will not be live until after DynamoDB loads the data.

Performing an online migration to DynamoDB by migrating each table 1:1

Many relational databases have a feature called Change Data Capture (CDC), where the database allows users to request a list of the changes to a table that happened before or after a specific point in time. CDC uses internal logs to enable this feature and it does not require the table to have any timestamp column to work.

When migrating a schema of SQL tables to a NoSQL database, you might want to combine and reshape your data into fewer tables. Doing so will allow you to collect data in a single place and avoid having to manually join related data in multi-step read operations. However, single table data shaping is not always required and sometimes you'll migrate tables 1-for-1 into DynamoDB. These 1-to-1 table migrations are less complicated as you can leverage the source database CDC feature, using common ETL tools that support this type of migration. The data for each row may still be transformed into new formats, but the scope of each table remains the same.

Consider migrating SQL tables 1-to-1 into DynamoDB, with the caveat that DynamoDB doesn't support server-side joins. You'll need to add logic to your application to combine data from multiple tables.

Plan

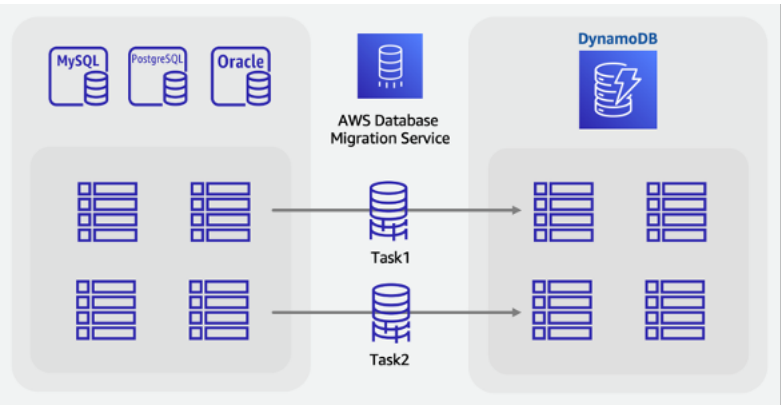
Perform an online migration of each table into DynamoDB using AWS DMS

Tools

- AWS DMS (DMS)

Online migration steps:

1. Identify the tables in your source schema that will be migrated
2. Create the same number of tables in DynamoDB with the same key structure as in the source
3. Create a replication server in AWS DMS and configure the source and target endpoints
4. Define any per-row transformations required (such as concatenated columns or conversion of dates to ISO-8601 string format)
5. Create a migration task for each table for **Full Load and Change Data Capture**
6. Monitor these tasks until the ongoing replication phase has begun
7. At this point you may perform any validation audits and then switch users to the application that reads and writes to DynamoDB



Perform an online migration to DynamoDB using a custom staging table

Like in the offline migration scenario above, you can choose to combine tables to leverage unique NoSQL access patterns (for example, transforming four legacy tables into one single DynamoDB table). A SQL `VIEW` could do the work within the source database to prepare a single dataset representing all four tables in one set.

However, for online migrations with live, changing data, you are unable to leverage CDC features as they aren't supported for `VIEW`s. If your tables include a last-updated timestamp column, and these are incorporated these into the `VIEW`, you can then build a custom ETL job that uses these to achieve a bulk load with synchronization.

A novel approach to this challenge is to use standard SQL features such as views, stored procedures, and triggers to create a new SQL table that is in the final desired DynamoDB NoSQL format.

If your database server has the spare capacity, it's possible to create this single staging table before migration begins. You would do this by writing a stored procedure that will read from existing tables, transform data as needed, and write to the new staging table. You can add a set of triggers to replicate changes in tables into the staging table in real time. If triggers are not allowed per company policy, changes to stored procedures can accomplish the same result. You would add a few lines of code to any procedure that writes data, to additionally write the same changes into the staging table.

Having this staging table in place that is fully synchronized with the legacy application tables will give you a great starting point for a live migration. Tools using database CDC to accomplish live migrations, such as AWS DMS, can now be used against this table. An advantage of this approach is that it uses well-known SQL skills and features available in the relational database engine.

Plan

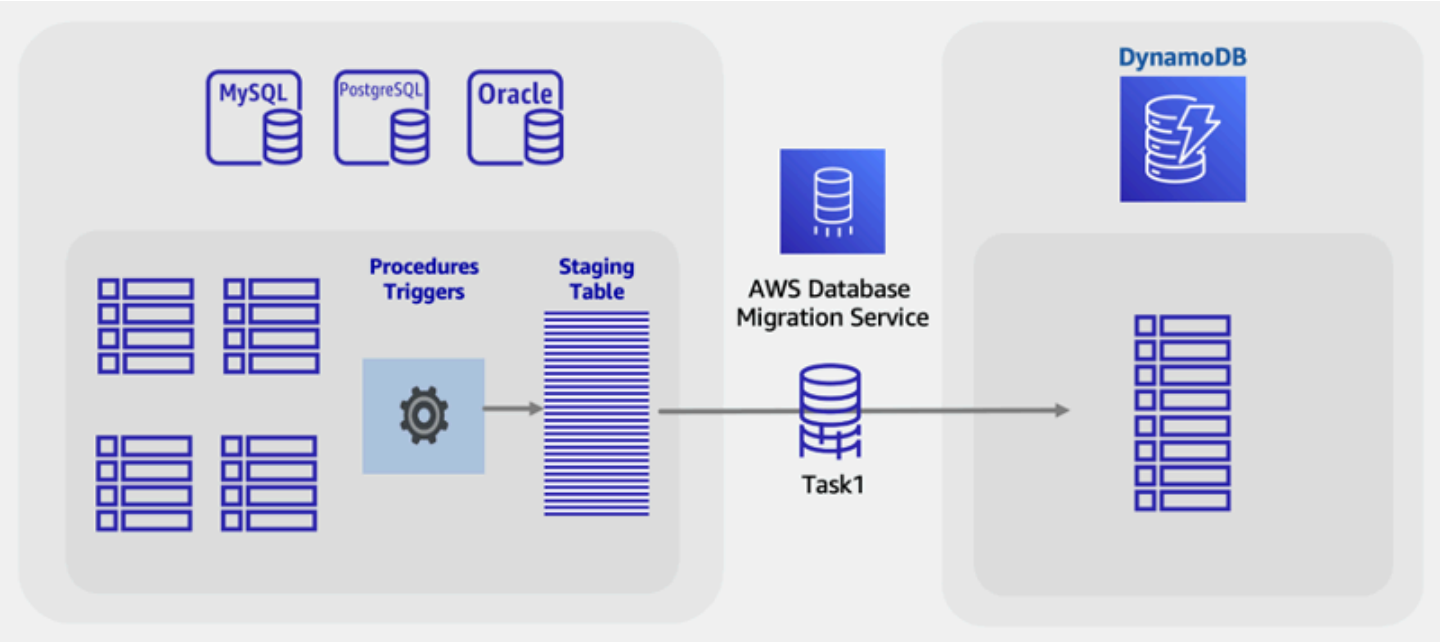
Perform an online migration with an SQL staging table using AWS DMS

Tools

- Custom SQL stored procedures or triggers
- AWS DMS

Online migration steps:

1. Within the source relational database engine, ensure there is some extra disk space and processing capacity.
2. Create a new staging table in the SQL database, with timestamps or CDC features enabled
3. Write and run a stored procedure to copy existing relational table data into the staging table
4. Deploy triggers or modify existing procedures to dual write into the new staging table while performing normal writes to existing tables
5. Run AWS DMS to migrate and synchronize this source table to a target DynamoDB table



This guide presented several considerations and approaches for migrating relational database data into DynamoDB, with a focus on minimizing downtime and using common database tools and techniques. For more information, see the following:

- AWS DMS User Guide
- AWS Glue User Guide
- Best Practices for Migrating from RDBMS to DynamoDB